

```

import { q, one, j } from '../db.js';
import { subscribe } from './events.js';

// Nobody tells the platform to update Facebook.
// A canonical value changes, and every installed app that carries that value
// gets a run queued against it. The owner never picks the destinations.
export async function fanOut({ ctx, key }) {
  const ws = await one(
    `select * from workspace where business_id = $1 and kind = 'cloud_android'`,
  );
  if (!ws) return { queued: [] };

  const apps = await q(
    `select da.package_key from device_app da
    where da.workspace_id = $1 and da.logged_in = true`, [ws.id]
  );

  const { request } = await import('./executor.js');
  const queued = [];

  for (const app of apps) {
    const map = await one(
      `select carries, routes from appmap where package_key = $1 and status = 'ac
      order by created_at desc limit 1`, [app.package_key]
    );
    if (!map) continue;
    const carries = j(map.carries) ?? [];
    if (!carries.includes(key)) continue;

    const out = await request({
      ctx, capability: 'channel.push', input: { packageKey: app.package_key, key
      resource: app.package_key,
    });
    queued.push({
      packageKey: app.package_key,
      state: out.execution?.state ?? 'error',
      verification: out.verification ?? null,
      error: out.error?.message ?? out.execution?.error ?? null,
    });
  }

  return { key, queued };
}

```

```

// Which canonical key a completed kernel write touched.
const KEY_OF = {
  'business.set_hours': 'hours',
  'business.set_temporary_closure': 'hours',
};

export function installFanOut() {
  subscribe('execution.succeeded', async ({ businessId, payload }) => {
    const key = KEY_OF[payload.capability];
    if (!key) return;
    const ctx = await systemContext(businessId);
    await fanOut({ ctx, key });
  });

  // A plain fact change carries its own key.
  subscribe('canonical.fact_changed', async ({ businessId, payload }) => {
    const ctx = await systemContext(businessId);
    await fanOut({ ctx, key: payload.key });
  });
}

// Fan-out runs as the platform on the owner's behalf, so it is not blocked by
// the acting person's grants. It is still recorded and still receipted.
async function systemContext(businessId) {
  const business = await one(`select * from business where id = $1`, [businessId])
  return {
    businessId, business, person: null,
    membership: { id: null, role: 'system' },
    system: true,
  };
}

```